



Smart Farming Decision Support System



A MINI PROJECT REPORT (60 AM 6P3)

Submitted by

PRAVEEN S (2303737714821030)

MEGANATHAN R (2303737714821022)

ARSHIYA NASIRIN M (2303737714822040)

in partial fulfillment of the requirement

for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE ENGINEERING

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

K.S. RANGASAMY COLLEGE OF TECHNOLOGY

(Autonomous)

TIRUCHENGODE – 637 215

APRIL 2026

**K.S. RANGASAMY COLLEGE OF TECHNOLOGY
TIRUCHENGODE - 637 215**

BONAFIDE CERTIFICATE

Certified that this Mini Project Report titled “**Smart Farming Decision Support System**” is the Bonafide work of **PRAVEEN S (2303737714821030)**, **MEGANATHAN R (2303737714821022)** AND **ARSHIYA NASIRIN M (2303737714822040)** who carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here in does not form part of any other project report or dissertation based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Mr.V. Thamizharasu., B.E, M.E.,
SUPERVISOR
Assistant Professor
Department of CSE (Artificial Intelligence and
Machine Learning)

SIGNATURE

Dr. C. Rajan, M.E., Ph.D.,
HEAD OF THE DEPARTMENT
Professor
Department of CSE (Artificial Intelligence and
Machine Learning)

DECLARATION

We jointly declare that the Mini Project Report on "**Smart Farming Decision Support System**" is the result of original work done by us and, to the best of our knowledge, similar work has not been submitted to "**ANNA UNIVERSITY, CHENNAI**" for the requirement of the Degree of Bachelor of Engineering. This project report is submitted in partial fulfilment of the requirement for the award of the Degree of Bachelor of Engineering.

Signature

PRAVEEN S

MEGANATHAN R

ARSHIYA NASIRIN M

Place: Tiruchengode

Date:

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our honorable Chairman **Thiru. R. SRINIVASAN, B.B.M.**, for providing immense facilities at our institution.

We would like to express our sincere gratitude to our Vice Chairman, **Thiru. K. S. SACHIN, MBA (UK)**., for providing a supportive environment for the completion of our project work.

We are truly grateful to express our gratitude to our beloved Principal **Dr. R. GOPALAKRISHNAN, M.E., Ph.D.**, for the encouragement given towards the progress and completion of our project.

We proudly render our immense gratitude to the Head of the Department **Dr. C. RAJAN, M.E., Ph.D.**, for his guidance and support in shaping the direction of this project. Your insightful feedback and expertise have been instrumental in refining our approach and methodology.

We are highly indebted to provide our heart full thanks to supervisor **Mr.V. THAMIZHARASU., B.E, M.E.**, Assistant Professor, for his valuable ideas, encouragement and supportive guidance throughout the project. Your constructive criticism and attention to detail have greatly contributed to the quality of this report.

We wish to extend our sincere thanks to all faculty members of Artificial Intelligence and Machine Learning for their valuable suggestions, kind co-operation and constant encouragement for successful completion of this project.

We wish to acknowledge the help received from various Departments, individuals, and the AICTE IDEA Lab facilities in the successful completion of this project.

ABSTRACT

Crop selection and fertilizer planning are critical decisions that directly influence yield stability, cost of inputs, and long-term soil health. In many practical settings, these decisions are made using experience and generalized seasonal guidelines, which may not fully account for measurable soil nutrients, weather patterns, and local management factors. This project presents a system titled "Smart Farming Decision Support System" that recommends a suitable crop and a supporting fertilizer plan from field conditions. The proposed system uses a machine learning pipeline to predict the most compatible crop class based on inputs such as Nitrogen, Phosphorus, Potassium, soil pH, soil moisture, organic matter, temperature, humidity, rainfall, sunlight hours, altitude, and contextual variables such as soil type, region, season, irrigation method, and state. A Random Forest classifier produces a predicted crop along with confidence scores and top alternative crops. In addition, a rule-based fertilizer recommendation layer compares the inputs with crop profile ranges to generate matched-condition explanations and attention points when values fall outside preferred bands. The system is delivered through a Flask web interface and a JSON API, enabling quick manual analysis and repeatable recommendations that support practical farm planning and sustainable resource use.

Keywords: Smart Farming, Decision Support System, Crop Recommendation, Fertilizer Planning, Random Forest, Feature Selection, Flask, Soil Nutrients, Weather Variables

This research contributes towards achieving Sustainable Development Goals (SDGs) 2 and 12, specifically focusing on Zero Hunger and Responsible Consumption and Production by enabling intelligent, soil-aware crop planning and efficient fertilizer use for sustainable agricultural systems.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO.
	ABSTRACT	v
	LIST OF TABLES	viii
	LIST OF FIGURES	ix
	LIST OF SYMBOLS AND ABBREVIATIONS	x
1	INTRODUCTION	1
2	LITERATURE REVIEW	2
	2.1 CONCLUSION FROM LITERATURE REVIEW	3
3	SYSTEM ANALYSIS	4
	3.1 EXISTING SYSTEM	4
	3.2 PROPOSED SYSTEM	4
	3.3 HARDWARE REQUIREMENTS	5
	3.4 SOFTWARE REQUIREMENTS	5
4	SYSTEM DESIGN	6
	4.1 DATA ACQUISITION	7
	4.2 PREPROCESSING	7
	4.3 RANDOM FOREST CROP RECOMMENDATION	7
	4.4 OUTPUT VISUALIZATION	9

5	RESULTS AND DISCUSSION	10
	5.1 OUTPUT SCREENSHOTS	11
6	CONCLUSION	13
	APPENDIX	14
	REFERENCES	17

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
4.1	Prediction Model Performance Metrics	8
4.2	Top Influential Features	8

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	System Architecture	6
5.1	Web Interface Output – Input Form	11
5.2	Web Interface Output – Recommendation Result	11

LIST OF SYMBOLS AND ABBREVIATIONS

ABBREVIATIONS

DSS	-	Decision Support System
ML	-	Machine Learning
RF	-	Random Forest
MI	-	Mutual Information
API	-	Application Programming Interface
CSV	-	Comma-Separated Values
N, P, K	-	Nitrogen, Phosphorus, Potassium (kg/ha)
pH	-	Soil acidity/alkalinity
Acc	-	Accuracy
Prec	-	Precision
Rec	-	Recall
F1	-	F1-score

CHAPTER 1

INTRODUCTION

Agriculture decisions are increasingly influenced by measurable field factors such as soil fertility, moisture availability, and climate variability. Selecting an unsuitable crop for a given soil and weather context can reduce yield, increase production costs, and raise risk for farmers. Similarly, applying fertilizers without matching crop requirements and nutrient status can cause nutrient imbalance, poor plant growth, and long-term soil degradation. These challenges are amplified by irregular rainfall patterns and temperature shifts, which make seasonal planning less predictable than in previous decades. Therefore, practical decision tools that combine soil and climate information into actionable recommendations are increasingly important for sustainable farming.

Traditional planning often relies on experience, local advice, and generalized cropping calendars. While such methods are valuable, they may not explicitly combine multiple measurable indicators such as N, P, K, pH, soil moisture, temperature, humidity, rainfall, and sunlight. As a result, two fields in the same region may require different crop choices and nutrient corrections, but those differences can be missed without structured analysis. A decision support system can convert such measurements into repeatable guidance, helping farmers compare options and plan inputs more effectively.

Machine learning models are well suited to this task because crop suitability is not determined by a single variable; it depends on nonlinear interactions among nutrients, climate, and contextual factors such as soil type, season, and irrigation method. In addition, presenting alternatives and confidence levels improves decision quality by reducing over-reliance on a single suggestion.

The main objective of this project is to develop a "Smart Farming Decision Support System" that takes practical field inputs and returns a crop recommendation with a fertilizer plan. The system integrates a Random Forest classifier for crop prediction with a rule-based fertilizer recommendation layer that explains matched conditions and flags attention points. A web interface and JSON API make the system easy to use for academic demonstration and for future integration with broader smart agriculture platforms.

CHAPTER 2

LITERATURE REVIEW

Research on precision agriculture and digital advisory systems emphasizes the value of combining soil, weather, and management information to improve farm decisions. Decision Support Systems (DSS) in agriculture have evolved from expert-rule tables to data-driven models as structured datasets and computing tools became widely available. Modern DSS designs typically focus on usability, interpretability, and reliability, because end users need actionable outputs rather than raw predictions. Within this context, crop recommendation can be framed as a multi-class classification problem where the target label is the most suitable crop given nutrient and climate conditions [5][6].

A common theme in the literature is that tree-based ensemble methods provide strong performance on tabular agronomic datasets. Random Forests, in particular, are robust to noise and can capture nonlinear relationships among inputs while still enabling feature-importance interpretation [1]. Practical implementations also stress the importance of consistent preprocessing: missing values, mixed numeric and categorical fields, and scale differences can introduce bias if not handled properly [2]. Feature selection techniques (including mutual information scoring) are frequently used to reduce redundancy after one-hot encoding and to keep pipelines compact. Standard evaluation practices include accuracy, precision, recall, F1-score, and confusion-matrix analysis to understand class overlap and stability across crops.

Beyond purely predictive models, the literature also highlights the role of interpretability and agronomic reasoning. Farmer-facing systems frequently combine predictive suitability models with deterministic rules for nutrient correction and advisories, because rule-based outputs are easier to explain and align with extension practices. Nutrient stewardship programs emphasize balanced management and the "Right source, Right rate, Right time, Right place" principle, which provides a practical framing for fertilizer recommendations [3]. Hybrid architectures therefore improve adoption by presenting both a recommendation and a rationale, including attention notes when an input is outside a preferred band.

Recent smart-farming research also explores integrating IoT sensors, remote sensing, and weather feeds to automate data capture and support continuous decision-making. Surveys on smart farming and big-data analytics report that combining heterogeneous data sources improves advisory quality but also raises challenges in data quality, calibration, and reliability of field measurements

[7][8]. For academic prototypes, a lightweight manual-input interface is often preferred because it is easy to test, demonstrates the pipeline clearly, and can later be extended to sensor inputs. Deployment-focused studies further show that web frameworks enable DSS tools to expose both human-friendly pages and machine-readable APIs, supporting integration with dashboards and mobile apps [4].

Studies on crop recommendation datasets further note that models must generalize across locations and seasons, which requires careful validation and consistent feature definitions. Data leakage and inconsistent units can inflate reported accuracy and reduce practical usefulness. Therefore, many implementations emphasize strict input validation, standardized preprocessing, and reporting class-wise performance to understand which crops are frequently confused. For fertilizer planning, research and extension material commonly recommend coupling nutrient corrections with explanations and caution notes, since farmers benefit from knowing which variables are outside preferred bands and what corrective actions can be taken before the next season.

Overall, the literature supports a pipeline-based approach in which data validation, preprocessing, model inference, and explainable output generation work together. This motivates the design of the Smart Farming Decision Support System as a hybrid ML + rule-based solution delivered through a web interface.

2.1 CONCLUSION FROM LITERATURE REVIEW

The literature indicates that crop recommendation benefits from models that can combine soil nutrients, climate variables, and local context into a single decision output. Ensemble classifiers such as Random Forest are widely used because they handle nonlinear interactions and provide interpretable feature importance. Effective DSS solutions also require robust preprocessing and clear evaluation using cross-validation and class-wise metrics. For fertilizer guidance, rule-based methods remain practical and explainable, especially when grounded in nutrient stewardship principles. Finally, usability is essential: presenting confidence and alternatives through a simple web interface improves trust and supports real-world decision-making. These findings motivate the hybrid architecture used in this project.

CHAPTER 3

SYSTEM ANALYSIS

EXISTING SYSTEM

In many existing farming workflows, crop selection and fertilizer use are guided by experience, local traditions, and generalized seasonal suggestions. While such knowledge is valuable, it often does not integrate measurable soil test indicators with changing weather patterns in a structured way. Farmers may select a crop that is popular in the region but not well matched to their soil nutrient balance, moisture availability, or irrigation method. Fertilizer plans are also commonly applied using fixed routines or vendor advice, which can lead to over-application (higher cost and environmental runoff) or under-application (low yield potential). Another limitation is that manual decision-making is difficult to document and evaluate: there is usually no quantitative validation, no confidence estimate, and no systematic way to compare alternatives. These gaps motivate a decision support system that can consistently process field inputs and provide explainable recommendations.

In addition, the existing approach rarely provides traceability for why a specific crop is selected, making it hard to learn from outcomes across seasons. When weather patterns deviate from expectations, decisions based on historical norms may fail, and farmers may not have a structured method to explore alternative crops that better match current conditions.

PROPOSED SYSTEM

The proposed Smart Farming Decision Support System provides a data-driven crop recommendation and fertilizer planning workflow. Users enter soil nutrients (N, P, K), pH, moisture, organic matter, and weather-context variables such as temperature, humidity, rainfall, sunlight hours, and altitude. They also select categorical context such as soil type, region, state, season, and irrigation method. After validation, a trained machine learning pipeline predicts the most suitable crop and returns confidence scores with top alternative crops. A rule-based fertilizer layer compares the inputs with crop profile ranges to generate matched-condition explanations and attention points, and suggests nutrient-correction fertilizers where appropriate.

The system is exposed through a Flask web interface and a JSON API for programmatic access, making it suitable for both demonstrations and future integration. Because the model and preprocessing are packaged as a single serialized pipeline, the same transformations are applied

consistently during training and prediction, which reduces human error and improves reproducibility of the recommendations.

HARDWARE REQUIREMENTS

The system can be developed and executed on a standard laptop/desktop with a multi-core processor (Intel i5 or above). A minimum of 8 GB RAM is recommended for comfortable data processing and model execution, while 16 GB RAM improves training speed during cross-validation. Storage needs are modest, but at least 1–2 GB of free disk space is suggested for the dataset, model artifact, and plots. GPU hardware is not required for Random Forest training or inference. For demonstrations, the Flask web app can run locally with a browser as the client, and internet connectivity is optional.

SOFTWARE REQUIREMENTS

The system is implemented in Python using open-source libraries. Flask is used for the web UI and JSON endpoints. Pandas and NumPy support dataset processing, and scikit-learn is used to build a Pipeline with preprocessing, feature selection, and a Random Forest classifier. Joblib is used to save and load the trained pipeline. Matplotlib and Seaborn are used for visualization of feature importance and confusion matrix. Development can be done in VS Code or similar IDEs, and dependencies are installed via a requirements file to ensure reproducibility. The application can be executed on Windows or Linux with a standard Python virtual environment.

CHAPTER 4

SYSTEM DESIGN

The system design outlines the workflow of the Smart Farming Decision Support System from input collection to recommendation output. User inputs are validated, preprocessed, and passed through a trained crop model to obtain ranked crop probabilities. A rule-based fertilizer module then compares inputs against crop profile ranges to produce a fertilizer plan with matched conditions and attention points. By keeping preprocessing and modeling inside a single pipeline and exposing results through both HTML pages and JSON endpoints, the design ensures consistency, low latency, and easy integration for future deployment scenarios.

The system consists of four main stages: data acquisition, preprocessing, prediction modeling, and output visualization. Each stage is designed to support transparency and repeatability of recommendations.

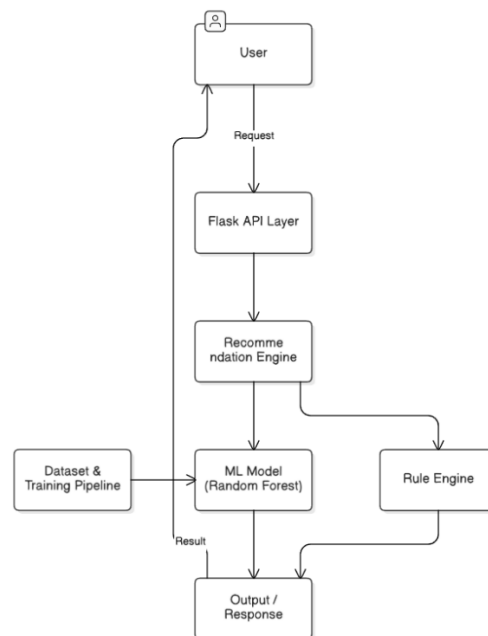


Fig. 4.1 System Architecture

Fig. 4.1 shows the end-to-end flow from input capture to preprocessing, model inference, and result delivery. The pipeline applies imputation, encoding, and feature selection, then predicts crop probabilities and ranks alternatives. A fertilizer rule engine adds matched-condition explanations and attention points before results are returned as HTML or JSON.

4.1 DATA ACQUISITION

Data acquisition includes preparing the crop recommendation dataset for training and collecting user inputs during prediction. The training dataset contains soil nutrients, weather variables, and contextual factors along with crop labels. Records are stored in CSV form for repeatable training and evaluation. At runtime, values are captured through a grouped web form or a JSON API request, enabling consistent feature alignment between training and inference. Input units and ranges are validated at runtime to reduce entry errors.

4.2 PREPROCESSING

Preprocessing prepares mixed-type inputs for classification. Numeric features use robust imputation so that missing values do not break predictions. Categorical features use imputation and one-hot encoding to convert selected options into model-ready indicator variables. After encoding, mutual-information based feature selection retains the most informative transformed features, reducing redundancy and improving pipeline efficiency. The same preprocessing steps are applied during training and inference through a single serialized pipeline artifact.

4.3 RANDOM FOREST CROP RECOMMENDATION

The core prediction stage applies a trained Random Forest classifier embedded in a single end-to-end pipeline. Training uses a stratified train-test split and k-fold cross-validation to estimate stability. During inference, the model outputs the predicted crop and probability scores for all crop classes. These probabilities are sorted to present the top alternatives with confidence percentages. A fertilizer recommendation layer then compares the same inputs to crop profile ranges and adds nutrient-correction suggestions, while generating matched-condition explanations and attention points. The chosen model settings balance accuracy and generalization while keeping inference fast for real-time web requests.

Table 4.1 Prediction Model Performance Metrics

Metric	Description	Value
Accuracy	Overall correctness of crop class prediction	96.09%
Precision (weighted)	Weighted precision across all crop classes	96.42%
Recall (weighted)	Weighted recall across all crop classes	96.15%
F1-Score (weighted)	Weighted harmonic mean of precision and recall	96.12%
CV Accuracy (mean)	Cross-validation mean accuracy (k-fold)	~95.25%
CV Std Dev	Cross-validation standard deviation	Low

Table 4.1 shows the performance of the prediction model evaluated using standard metrics. The results indicate that the system provides high prediction accuracy along with strong recall, ensuring that actual crop classes are reliably identified. The cross-validation mean confirms consistent performance across folds, indicating the pipeline generalizes well to unseen inputs.

Table 4.2 Top Influential Features

Rank	Feature	Description
1	Nitrogen (kg/ha)	Primary macronutrient; high MI with crop class
2	Rainfall (mm)	Annual rainfall; strongly differentiates water-intensive crops
3	Sunlight Hours	Photoperiod sensitivity of crop types
4	Potassium (kg/ha)	Differentiates root and cereal crops
5	Soil Moisture (%)	Available water content at field time of measurement
6	Humidity (%)	Affects disease pressure and crop tolerance
7	Temperature (°C)	Thermal suitability window per crop class
8	Altitude (m)	Elevation effects on temperature and crop range
9	Phosphorus (kg/ha)	Root development and flowering support

Table 4.2 shows the top features ranked by mutual information score. Nitrogen, rainfall, and sunlight hours are the most influential predictors, consistent with agronomic knowledge about crop nutrient and climate requirements.

4.4 OUTPUT VISUALIZATION

In the final stage, results are presented through a Flask web interface. The result page displays the recommended crop, confidence score, fertilizer plan, top alternatives, matched conditions, and caution notes. An input summary is shown for verification. The same recommendation structure is available through a JSON API endpoint, which supports integration with external applications and enables automated testing of the decision workflow. Clear error messages are returned when inputs are missing or out of valid bounds.

CHAPTER 5

RESULTS AND DISCUSSION

The execution of this project resulted in the successful development of a crop and fertilizer decision support system that transforms field measurements into actionable recommendations. The machine learning pipeline demonstrates strong multi-class classification performance on the prepared dataset and produces stable results under cross-validation. The Random Forest classifier performs well because it captures nonlinear relationships between nutrients and climate variables, and it remains robust to moderate input variability. Feature importance analysis indicates that nitrogen, rainfall, sunlight hours, potassium, soil moisture, humidity, temperature, altitude, and phosphorus are influential predictors, which aligns with agronomic expectations.

On the evaluation split, the model achieves approximately 96.09% accuracy, with weighted precision of about 96.42% and weighted F1-score of about 96.12%. Cross-validation accuracy mean is approximately 95.25% with a low standard deviation, suggesting consistent performance across splits. The confusion matrix shows that most crop classes are well separated, with remaining misclassifications concentrated among crops with overlapping environmental bands. This motivates the inclusion of top alternative crops and confidence percentages, allowing users to treat near-ties as practical options depending on market and resource constraints.

A key outcome of the project is the hybrid output: the system not only recommends a crop but also provides a fertilizer plan and flags attention points when inputs deviate from preferred crop bands. Matched-condition explanations increase transparency, and caution notes encourage users to review out-of-range values instead of blindly trusting a single prediction. Because the current dataset is bounded and structured, future evaluation on real soil-test and weather observations is recommended for field deployment. Nevertheless, the results confirm that the proposed pipeline and interface form a reliable, presentation-ready decision support prototype.

5.1 OUTPUT SCREENSHOTS

Fig. 5.1 Web Interface Output - Input Form

Fig. 5.1 shows the home page of the Smart Farming Decision Support System. The interface groups inputs into soil nutrients, weather profile, and location/management fields. Each numeric field displays units and valid ranges to reduce entry errors. The page also presents model statistics such as dataset size, number of crop classes, and validation accuracy, which improves trust and supports explanation during demonstrations.

Fig. 5.2 Web Interface Output - Recommendation Result

Fig. 5.2 shows the result page after submitting field inputs. The system displays the recommended crop with a confidence percentage, a suggested fertilizer plan, and a ranked list of alternative crops. Matched-condition bullets summarize which inputs align with the crop profile, while attention points highlight where values are outside preferred bands. An input summary is also displayed so users can verify the exact values used for the recommendation.

CHAPTER 6

CONCLUSION

Machine learning and decision support tools can improve farm planning by converting measurable soil and climate conditions into consistent recommendations. The Smart Farming Decision Support System demonstrates a complete end-to-end workflow for crop recommendation and fertilizer planning. The system validates practical field inputs, applies a reproducible preprocessing and classification pipeline, and returns a predicted crop with confidence ranking. The addition of a rule-based fertilizer component improves interpretability by generating matched conditions and highlighting attention points for out-of-band values. Evaluation results show strong predictive performance and stable cross-validation behavior, making the prototype suitable for academic demonstration.

Despite the strong results, the system's reliability in real-world deployment would depend on continued validation with field-collected soil tests and local weather observations. The dataset used in this project is structured and bounded; extreme or novel conditions outside the training distribution may reduce confidence. Future work can include integration with live weather APIs and soil sensors, expansion of crop varieties for additional regions, and addition of explainability views that show feature contribution for each prediction. A mobile-first interface, persistent storage, and user profiles could further improve adoption. Overall, the project shows that a hybrid ML + rule-based approach can provide practical, explainable guidance for smarter farming decisions.

The present work is therefore a strong foundation for a larger smart-agriculture platform where data is continuously collected, recommendations are tracked over time, and models are retrained using verified outcomes to improve long-term reliability and regional relevance.

APPENDIX

```
# -----
# PROJECT MODULES (REFERENCE SNIPPETS)
# -----

# 1) BASIC PACKAGE INSTALLATION
pip install -r requirements.txt

# 2) DATASET GENERATION (CSV)
python src/generate_dataset.py

# 3) TRAINING AND EVALUATION
python src/train_model.py

# 4) RUN THE WEB APPLICATION
python app.py

# -----
# FLASK ENDPOINTS (OVERVIEW)
# -----

GET / -> Input form
POST /predict -> HTML result page
POST /api/predict -> JSON recommendation
GET /api/health -> health + model availability

# -----
# INPUT FEATURES (SCHEMA)
# -----

NUMERIC: nitrogen_kg_ha, phosphorus_kg_ha, potassium_kg_ha, temperature_c,
humidity_pct,
        soil_ph, rainfall_mm, soil_moisture_pct, organic_matter_pct,
sunlight_hours, altitude_m
CATEGORICAL: soil_type, region, state, season, irrigation_method

# -----
# PIPELINE BUILDING (SKLEARN)
# -----
```



```

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.feature_selection import SelectPercentile, mutual_info_classif
from sklearn.ensemble import RandomForestClassifier

def build_pipeline(numeric_cols, categorical_cols):
    numeric_pipeline = Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="median")),
    ])
    categorical_pipeline = Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore",
sparse_output=False)),
    ])
    preprocessor = ColumnTransformer(transformers=[
        ("numeric", numeric_pipeline, numeric_cols),
        ("categorical", categorical_pipeline, categorical_cols),
    ])
    selector = SelectPercentile(score_func=mutual_info_classif, percentile=75)
    model = RandomForestClassifier(
        n_estimators=260, max_depth=14, min_samples_split=4,
        min_samples_leaf=2, class_weight="balanced_subsample",
        random_state=42, n_jobs=-1,
    )
    return Pipeline(steps=[
        ("preprocess", preprocessor),
        ("select", selector),
        ("model", model),
    ])

# -----
# FERTILIZER RULE (EXAMPLE)
# -----

NUMERIC_CORRECTION_MAP = {
    "nitrogen_kg_ha": "Urea",
    "phosphorus_kg_ha": "DAP",
    "potassium_kg_ha": "MOP",
}

```

```

# If nutrient is below preferred crop band -> add correction fertilizer.
# If within band -> add matched-condition explanation.

# -----
# API PAYLOAD EXAMPLE (/api/predict)
# -----

{
  "nitrogen_kg_ha": 85, "phosphorus_kg_ha": 48, "potassium_kg_ha": 55,
  "temperature_c": 27, "humidity_pct": 68, "soil_ph": 6.8,
  "rainfall_mm": 950, "soil_moisture_pct": 42, "organic_matter_pct": 2.2,
  "sunlight_hours": 8.5, "altitude_m": 350, "soil_type": "loam",
  "region": "central", "state": "Madhya Pradesh",
  "season": "kharif", "irrigation_method": "canal"
}

# -----
# OUTPUT ARTIFACTS
# -----

data/crop_recommendation_dataset.csv
models/crop_recommender_pipeline.joblib
models/training_metrics.json
reports/classification_report.txt
reports/feature_importance.csv
reports/confusion_matrix.csv

```

REFERENCES

- [1] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [3] International Plant Nutrition Institute (IPNI), "4R Nutrient Stewardship: Right Source, Right Rate, Right Time, Right Place," guidance documents, accessed 2026.
- [4] Pallets Projects, "Flask Documentation," <https://flask.palletsprojects.com/>, accessed 2026.
- [5] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009.
- [6] Q. Zhang, *Precision Agriculture Technology for Crop Farming*, CRC Press, 2016.
- [7] G. Wolfert, L. Ge, C. Verdouw, and M.-J. Bogaardt, "Big Data in Smart Farming – A review," *Agricultural Systems*, vol. 153, pp. 69–80, 2017.
- [8] G. McBratney, B. Whelan, T. Ancev, and J. Bouma, "Future directions of precision agriculture," *Precision Agriculture*, 2005.
- [9] M. I. Jordan and T. M. Mitchell, "Machine learning: Trends, perspectives, and prospects," *Science*, vol. 349, no. 6245, pp. 255–260, 2015.
- [10] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly Media, 2nd ed., 2019.
- [11] W. McKinney, "Data Structures for Statistical Computing in Python," *Proc. SciPy*, 2010.
- [12] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.
- [13] Matplotlib Development Team, "Matplotlib Documentation," <https://matplotlib.org/>, accessed 2026.
- [14] Seaborn Development Team, "Seaborn Documentation," <https://seaborn.pydata.org/>, accessed 2026.
- [15] Scikit-learn Developers, "User Guide: Model evaluation and selection; RandomForestClassifier," <https://scikit-learn.org/>, accessed 2026.
- [16] Food and Agriculture Organization of the United Nations (FAO), reports on sustainable agriculture, soil health, and digital advisory systems, accessed 2026.
- [17] Government of India, "Soil Health Card Scheme," Ministry of Agriculture & Farmers Welfare (MoAFW) guidelines and resources, accessed 2026.
- [18] USDA Natural Resources Conservation Service (NRCS), "Soil Health" technical notes and publications, accessed 2026.

- [19] FAO, "Sustainable soil management and fertilizer use" technical guides and reports, accessed 2026.
- [20] World Meteorological Organization (WMO), guidance on meteorological data quality and climate services, accessed 2026.
- [21] India Meteorological Department (IMD), climate summaries and seasonal rainfall reports, accessed 2026.
- [22] R. Kohavi, "A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection," IJCAI, 1995.
- [23] C. Molnar, "Interpretable Machine Learning," online book, accessed 2026.
- [24] World Bank, "Climate-Smart Agriculture Sourcebook," guidance for resilient, sustainable farming, accessed 2026.
- [25] USDA, "Fertilizer and nutrient management best practices," technical resources and decision-support guidance, accessed 2026.